CS-471: Machine Learning

Week 4 — Logistic Regression Continued, Multiclass Classification, Model Evaluation & Bias/Variance

Instructor: Dr. Hashir Kiani Sources: Lectures 4-1 & 4-2

Comprehensive Study Notes

Week 4

Contents

1 Logistic Regression — Worked Example (Lecture 4-1) 2
- 1.1 Quick Recap of the Formulas . . . 2
- 1.2 Training Dataset . . . 2

2 Multiclass Classification 3
- 2.1 The Problem . . . 3
- 2.2 One-vs-Rest (OvR / OvA) . . . 4
- 2.3 One-vs-One (OvO) . . . 4

3 Multinomial Logistic Regression 5
- 3.1 Motivation . . . 5
- 3.2 Weight Matrix $\mathbf{W}$ . . . 5
- 3.3 Input Matrix $\mathbf{X}$ . . . 6
- 3.4 Score Matrix $\mathbf{S}$ . . . 6
- 3.5 Probability Matrix $\mathbf{P}$ — The Softmax Function . . . 6
- 3.6 Loss Function — Categorical Cross Entropy . . . 7
- 3.7 Gradient and Update Rule . . . 7

4 Classification Model Performance Evaluation 8
- 4.1 The Confusion Matrix . . . 8
- 4.2 Accuracy . . . 9
- 4.3 Precision . . . 9
- 4.4 Recall (Sensitivity / True Positive Rate) . . . 9
- 4.5 F1 Score . . . 9
- 4.6 Specificity (True Negative Rate) and FPR . . . 10
- 4.7 Summary of All Metrics . . . 10
- 4.8 ROC Curve . . . 11
- 4.9 AUC (Area Under the ROC Curve) . . . 11

5 Underfitting and Overfitting 12
- 5.1 Definitions . . . 12

5.2 Underfitting 12
5.3 Overfitting 12
5.4 Bias and Variance 13
5.5 Regression Example (Polynomial Fitting) 14
5.6 Classification Example 14

6 Practice Questions **15**

7 Key Takeaways — Week 4 **16**

1 Logistic Regression — Worked Example (Lecture 4-1)

1.1 Quick Recap of the Formulas

Before the worked example, the lecture revisits the core logistic regression formulas.

Loss Function (Binary Cross Entropy):

$$\text{Loss}(y,\ h(\mathbf{x})) = -y\log\big(h(\mathbf{x})\big) - (1-y)\log\big(1-h(\mathbf{x})\big)$$

Average Training Loss:

$$\text{TrainLoss}(\mathbf{w}) = -\frac{1}{N}\sum_{i=1}^{N}\Big[y^{(i)}\log h(\mathbf{x}^{(i)}) + (1-y^{(i)})\log(1-h(\mathbf{x}^{(i)}))\Big]$$

Gradient Descent Update:

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} - \frac{\alpha}{N}\sum_{i=1}^{N}\big[h(\mathbf{x}^{(i)}) - y^{(i)}\big]\,\mathbf{x}^{(i)}$$

1.2 Training Dataset

The lecture uses the following three-point dataset (labels rewritten to the $y \in \{0,1\}$ convention for logistic regression):

Example	$\mathbf{x}$	y
1	$[0,\ 2]^\top$	1
2	$[-2,\ 0]^\top$	1
3	$[1,\ -1]^\top$	0

Feature extractor with bias term: $\boldsymbol{\phi}(\mathbf{x}) = [1,\ x_1,\ x_2]^\top$

Example

Full Single-Step Worked Example
Initialisation: $\mathbf{w} = [0,\ 0,\ 0]^\top$, learning rate $\alpha = 0.5$.

Step 1 — Compute scores $z^{(i)} = \mathbf{w}^\top\boldsymbol{\phi}(\mathbf{x}^{(i)})$:
All weights are zero initially so every score $= 0$.
Step 2 — Compute sigmoid outputs $h^{(i)} = g(z^{(i)}) = 1/(1+e^{-z^{(i)}})$:

$$h^{(1)} = h^{(2)} = h^{(3)} = g(0) = 0.5$$

Step 3 — Compute errors $e^{(i)} = h^{(i)} - y^{(i)}$:

$$e^{(1)} = 0.5 - 1 = -0.5$$
$$e^{(2)} = 0.5 - 1 = -0.5$$
$$e^{(3)} = 0.5 - 0 = +0.5$$

Step 4 — Compute gradient $\nabla = \frac{1}{N}\sum e^{(i)}\boldsymbol{\phi}^{(i)}$:

$$\nabla_{w_0} = \tfrac{1}{3}[(-0.5)(1) + (-0.5)(1) + (0.5)(1)] = \tfrac{-0.5}{3} \approx -0.167$$
$$\nabla_{w_1} = \tfrac{1}{3}[(-0.5)(0) + (-0.5)(-2) + (0.5)(1)] = \tfrac{0+1+0.5}{3} = 0.5$$
$$\nabla_{w_2} = \tfrac{1}{3}[(-0.5)(2) + (-0.5)(0) + (0.5)(-1)] = \tfrac{-1+0-0.5}{3} = -0.5$$

Step 5 — Update weights ($\alpha = 0.5$):

$$w_0 \leftarrow 0 - 0.5(-0.167) = +0.083$$
$$w_1 \leftarrow 0 - 0.5(0.5) = -0.25$$
$$w_2 \leftarrow 0 - 0.5(-0.5) = +0.25$$

$\mathbf{w}^{(1)} = [0.083,\ -0.25,\ +0.25]^\top$

Verification after one update:
Scores and predictions:

$$z^{(1)} = 0.083(1) + (-0.25)(0) + (0.25)(2) = 0.583, \quad h^{(1)} = g(0.583) \approx 0.64 \Rightarrow y = 1 \checkmark$$
$$z^{(2)} = 0.083(1) + (-0.25)(-2) + (0.25)(0) = 0.583, \quad h^{(2)} \approx 0.64 \Rightarrow y = 1 \checkmark$$
$$z^{(3)} = 0.083(1) + (-0.25)(1) + (0.25)(-1) = -0.417, \quad h^{(3)} \approx 0.40 \Rightarrow y = 0 \checkmark$$

All three examples are classified correctly after a single gradient step.

2 Multiclass Classification

2.1 The Problem

Binary classifiers output $y \in \{0, 1\}$ or $y \in \{-1, +1\}$. Many real-world problems have $K > 2$ classes (e.g. handwritten digit recognition: $K = 10$, image classification: thousands of classes). We need strategies to extend binary classifiers to K classes.

Two classical strategies:

1. **One-vs-Rest (OvR)** — also called One-vs-All (OvA)
2. **One-vs-One (OvO)**

2.2 One-vs-Rest (OvR / OvA)

Key Idea

Idea: For K classes, train K separate binary classifiers. Classifier k asks: "Is this example class k or not?"
Prediction: Choose the class whose classifier gives the *highest confidence score / probability*.

Training procedure:

1. For each class $k \in \{1, 2, \ldots, K\}$:
   (a) Relabel the dataset: examples from class k get label $+1$; all other examples get label -1 (or 0).
   (b) Train a binary classifier f_k on this modified dataset.
   (c) Each classifier is trained on the *entire* dataset.
2. At inference: compute $f_1(\mathbf{x}),\ f_2(\mathbf{x}),\ \ldots,\ f_K(\mathbf{x})$ and return $\hat{y} = \arg\max_k f_k(\mathbf{x})$.

Number of classifiers: K.

Class imbalance caveat: If there are 1000 examples total and 10 classes ($\approx$100 per class), each binary classifier sees $\sim$900 "negative" examples and only $\sim$100 "positive" examples. This class imbalance can degrade classifier quality.

Example

OvR on handwritten digits (10 classes, K=10):
Classifier f_3: trained to distinguish digit "3" vs. all others.
At inference on a new image: if f_3 gives score 0.85, f_7 gives 0.60, and all others give < 0.50, then predict class 3.

2.3 One-vs-One (OvO)

Key Idea

Idea: Train one binary classifier for every pair of classes. Classifier (j, k) distinguishes class j from class k.
Prediction: Each classifier "votes" for one class; the class with the *most votes* wins.

Training procedure:

1. For every pair (j, k) with $j < k$:
   (a) Extract only the examples belonging to class j or k.
   (b) Train a binary classifier f_{jk} on this subset.
2. At inference: run all $\frac{K(K-1)}{2}$ classifiers; tally votes; return the class with the most votes.

Number of classifiers: $\dfrac{K(K-1)}{2}$.

Property	One-vs-Rest	One-vs-One
No. of classifiers	K	$K(K-1)/2$
Training set per classifier	Full dataset	Subset (2 classes only)
Class imbalance	High	Low
Inference cost	K evaluations	$K(K-1)/2$ evaluations
Scales to large K?	Better	Worse (quadratic)
Generally preferred?	Often yes	When K is moderate

Real-Life Analogy (OvO): Think of a round-robin sports tournament. Every team plays every other team exactly once. The team that wins the most matches overall is declared champion. OvO is exactly that — each pair of classes "plays a match" and the class that beats the most others wins.

3 Multinomial Logistic Regression

3.1 Motivation

OvR and OvO are *reduction strategies* — they reuse existing binary classifiers. A cleaner, unified approach is **Multinomial Logistic Regression** (also called *softmax regression*), which directly models the probability of each class using a single model with a **weight matrix** instead of a weight vector.

3.2 Weight Matrix W

In binary logistic regression each class did not need its own weight vector (positive/negative classes share one hyperplane). For K classes, each class k gets a dedicated weight vector $\mathbf{w}_k \in \mathbb{R}^{d+1}$ (including the bias).

These are stacked column-wise into a weight matrix:

$$\mathbf{W} = \begin{bmatrix} \mathbf{w}_1 & \mathbf{w}_2 & \cdots & \mathbf{w}_K \end{bmatrix}$$

$$\text{Order of } \mathbf{W}: \quad (d+1) \times K$$

where d = number of input features and K = number of classes.

3.3 Input Matrix X

Training examples are stacked row-wise into an input (design) matrix:

$$\mathbf{X} = \begin{bmatrix} \boldsymbol{\phi}(\mathbf{x}^{(1)})^\top \\ \boldsymbol{\phi}(\mathbf{x}^{(2)})^\top \\ \vdots \\ \boldsymbol{\phi}(\mathbf{x}^{(N)})^\top \end{bmatrix}$$

$$\text{Order of } \mathbf{X}: \quad N \times (d+1)$$

where N = number of training examples in the batch. Each row $\mathbf{x}^{(i)}$ is the $(d+1)$-dimensional feature vector (with bias 1 prepended).

3.4 Score Matrix S

The raw class scores for all examples are computed in one matrix multiplication:

$$\boxed{\mathbf{S} = \mathbf{X}\,\mathbf{W}} \qquad \text{Order: } N \times K$$

Row i of $\mathbf{S}$ contains the K raw scores for training example i:

$$\mathbf{S}_{i,:} = \boldsymbol{\phi}(\mathbf{x}^{(i)})^\top \mathbf{W} = [s_{i1},\ s_{i2},\ \ldots,\ s_{iK}]$$

$s_{ik} = \mathbf{w}_k^\top \boldsymbol{\phi}(\mathbf{x}^{(i)})$ is the score of example i belonging to class k.

3.5 Probability Matrix P — The Softmax Function

Raw scores are converted to probabilities via the **softmax** function, applied row-wise:

$$\boxed{\mathbf{P} = \text{softmax}(\mathbf{S})} \qquad \text{Order: } N \times K$$

For example i and class k:

$$\mathbf{P}_{ik} = \frac{e^{s_{ik}}}{\sum_{j=1}^{K} e^{s_{ij}}}$$

Properties of softmax:

- All probabilities are positive: $\mathbf{P}_{ik} > 0$.
- Rows sum to 1: $\sum_{k=1}^{K} \mathbf{P}_{ik} = 1$ (valid probability distribution over classes).
- Large score $\Rightarrow$ high probability (exponential amplification).
- Softmax is the multi-class generalisation of sigmoid.

Key Idea

Softmax vs. Sigmoid: When $K = 2$, softmax reduces exactly to the sigmoid function. $\text{softmax}([s_1, s_2])_1 = \frac{e^{s_1}}{e^{s_1}+e^{s_2}} = \frac{1}{1+e^{s_2-s_1}}$. Setting $s_2 = 0$ (absorb into bias) recovers $g(s_1) = 1/(1 + e^{-s_1})$.

Real-Life Analogy: Imagine four restaurants ($K = 4$) competing for your dinner choice. You rate each one with a score. Softmax converts these raw scores into "probability you'll choose each restaurant" — a higher score means exponentially more likely to be chosen, and all probabilities add to 1.

3.6 Loss Function — Categorical Cross Entropy

Let $\mathbf{Y}$ be the one-hot encoded true label matrix (order $N \times K$), where $\mathbf{Y}_{ik} = 1$ if example i belongs to class k, and 0 otherwise.

$$\boxed{\text{TrainLoss}(\mathbf{W}) = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k=1}^{K}\mathbf{Y}_{ik}\,\log\big(\mathbf{P}_{ik}\big)}$$

Because $\mathbf{Y}_{ik}$ is 0 for all classes except the true class, only one term per example is non-zero:

$$\text{TrainLoss} = -\frac{1}{N}\sum_{i=1}^{N}\log\big(\mathbf{P}_{i,\,y^{(i)}}\big)$$

This is the negative log-probability of the true class. A perfect model has $\mathbf{P}_{i,y^{(i)}} \to 1$, pushing TrainLoss $\to 0$.

Comparison with binary cross entropy:

	Binary ($K = 2$)	Multiclass ($K > 2$)
Labels	$y \in \{0, 1\}$	One-hot $\mathbf{Y}$
Output	Sigmoid $h \in (0, 1)$	Softmax row $\mathbf{P}_{i,:}$
Loss	$-y \log h - (1 - y)\log(1 - h)$	$-\sum_k Y_{ik} \log P_{ik}$

3.7 Gradient and Update Rule

The gradient of the categorical cross-entropy training loss with respect to $\mathbf{W}$ has an elegant closed form:

$$\boxed{\nabla_{\mathbf{W}}\,\text{TrainLoss} = \frac{1}{N}\mathbf{X}^\top(\mathbf{P} - \mathbf{Y})} \qquad \text{Order: } (d+1) \times K$$

Update Rule:

$$\mathbf{W}_{\text{new}} = \mathbf{W}_{\text{old}} - \frac{\alpha}{N}\mathbf{X}^\top(\mathbf{P} - \mathbf{Y})$$

Key Idea

Structural elegance:
Compare the multinomial update to binary logistic regression:
Binary: $\mathbf{w} \leftarrow \mathbf{w} - \frac{\alpha}{N}\mathbf{X}^\top(\mathbf{h} - \mathbf{y})$
Multinomial: $\mathbf{W} \leftarrow \mathbf{W} - \frac{\alpha}{N}\mathbf{X}^\top(\mathbf{P} - \mathbf{Y})$
Identical structure — only $\mathbf{h}/\mathbf{y}$ are replaced by their matrix counterparts $\mathbf{P}/\mathbf{Y}$.

Example

Toy Multinomial LR Example ($K = 3$, $N = 2$, $d = 2$)
Suppose $\mathbf{S}$ for two examples is:

$$\mathbf{S} = \begin{bmatrix} 2 & 1 & 0 \\ 0 & 3 & 1 \end{bmatrix}$$

Softmax for example 1 (row 1): $Z_1 = e^2 + e^1 + e^0 = 7.389 + 2.718 + 1.000 = 11.107$

$$\mathbf{P}_{1,:} = \left[\frac{7.389}{11.107}, \frac{2.718}{11.107}, \frac{1.000}{11.107}\right] \approx [0.665,\ 0.245,\ 0.090]$$

Prediction: class 1 (highest probability 66.5%).
Softmax for example 2 (row 2): $Z_2 = e^0 + e^3 + e^1 = 1 + 20.086 + 2.718 = 23.804$

$$\mathbf{P}_{2,:} \approx [0.042,\ 0.844,\ 0.114]$$

Prediction: class 2 (highest probability 84.4%).

4 Classification Model Performance Evaluation

4.1 The Confusion Matrix

A **confusion matrix** is a table that summarises the performance of a binary classifier by cross-tabulating true labels against predicted labels.

	Predicted	
	Positive	**Negative**
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Definitions:

- **TP** (True Positive): Model says positive, actual is positive. Correct.
- **TN** (True Negative): Model says negative, actual is negative. Correct.
- **FP** (False Positive): Model says positive, actual is negative. Wrong. Also called

Type I error.

- **FN** (False Negative): Model says negative, actual is positive. Wrong. Also called *Type II error.*

4.2 Accuracy

$$\boxed{\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN}}$$

Proportion of all predictions that are correct.

Common Mistake / Pitfall

Accuracy is misleading with class imbalance!
Example: 990 healthy patients and 10 diseased patients. A model that always predicts "healthy" achieves 990/1000 = 99% accuracy, yet it catches *zero* disease cases. Use precision, recall, and F1 in such scenarios.

4.3 Precision

$$\boxed{\text{Precision} = \frac{TP}{TP + FP}}$$

"Of all the examples my model labelled positive, what fraction are actually positive?"

Use when: False positives are costly (e.g. spam filtering — better to miss a few spam emails than to delete important ones).

4.4 Recall (Sensitivity / True Positive Rate)

$$\boxed{\text{Recall} = \text{TPR} = \text{Sensitivity} = \frac{TP}{TP + FN}}$$

"Of all the actual positive examples, what fraction did my model detect?"

Use when: False negatives are costly (e.g. cancer detection — missing a cancer case is far more dangerous than a false alarm).

4.5 F1 Score

$$\boxed{F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}}$$

The F1 score is the **harmonic mean** of precision and recall.

- Ranges from 0 (worst) to 1 (best).
- High F1 requires both precision *and* recall to be high.
- Penalises extreme imbalance between the two (unlike arithmetic mean).

Key Idea

Why harmonic mean and not arithmetic mean?
If precision = 1.0 and recall = 0.0: Arithmetic mean = 0.5 (looks OK but one metric is 0!). Harmonic mean = 0 (correctly reflects failure). The harmonic mean is always $\leq$ the arithmetic mean, and is lower when the two values differ greatly.

4.6 Specificity (True Negative Rate) and FPR

$$\text{Specificity} = \text{TNR} = \frac{TN}{TN + FP}$$

"Of all actual negatives, what fraction did the model correctly reject?"

$$\text{FPR (False Positive Rate)} = 1 - \text{Specificity} = \frac{FP}{FP + TN}$$

Also: FNR (False Negative Rate) $= 1 - \text{Recall} = \frac{FN}{FN+TP}$.

4.7 Summary of All Metrics

Metric	Formula	Range	Higher is
Accuracy	$(TP + TN)/(TP + FP + FN + TN)$	$[0, 1]$	Better
Precision	$TP/(TP + FP)$	$[0, 1]$	Better
Recall / TPR	$TP/(TP + FN)$	$[0, 1]$	Better
Specificity / TNR	$TN/(TN + FP)$	$[0, 1]$	Better
FPR	$FP/(FP + TN)$	$[0, 1]$	Worse
FNR	$FN/(FN + TP)$	$[0, 1]$	Worse
F1 Score	$2 \cdot P \cdot R \,/\, (P + R)$	$[0, 1]$	Better

Example

Confusion Matrix Numerical Example
Suppose a COVID test gave the following results on 200 patients:

	Predicted	
	Positive	Negative
Actual Positive	TP = 80	FN = 20
Actual Negative	FP = 10	TN = 90

$$\begin{aligned}
\text{Accuracy} &= (80 + 90)/200 = 85\% \\
\text{Precision} &= 80/(80 + 10) = 88.9\% \\
\text{Recall} &= 80/(80 + 20) = 80.0\% \\
F_1 &= 2(0.889)(0.800)/(0.889 + 0.800) = 84.2\%
\end{aligned}$$

Interpretation: The test misses 20% of actual positive cases (recall = 80%), which could be dangerous in a medical context. Even though accuracy is 85%, for disease detection we care most about recall.

4.8 ROC Curve

The **ROC (Receiver Operating Characteristic) curve** plots the **True Positive Rate (Recall)** on the y-axis against the **False Positive Rate** on the x-axis as the classification *threshold* varies from 0 to 1.

Why vary the threshold? A logistic regression model outputs a probability $h(\mathbf{x})$. By default, we predict positive if $h > 0.5$. But we can adjust this threshold:

- Lower threshold ($\tau = 0.3$): classify more examples as positive $\Rightarrow$ higher recall, lower precision.
- Higher threshold ($\tau = 0.8$): classify fewer examples as positive $\Rightarrow$ lower recall, higher precision.

Each threshold value gives one (FPR, TPR) point; the ROC curve traces all such points.

Key reference points:

- (0, 0): threshold = 1 (classify everything as negative).
- (1, 1): threshold = 0 (classify everything as positive).
- (0, 1): perfect classifier (TPR = 1, FPR = 0).
- Diagonal line (TPR = FPR): random classifier.

4.9 AUC (Area Under the ROC Curve)

$$\text{AUC} \in [0, 1] \qquad \text{Higher is better; } 1 = \text{perfect, } 0.5 = \text{random}$$

AUC provides a *single-number* summary of classifier performance across all thresholds. A higher AUC means the model has a better ability to discriminate between positive and negative classes regardless of threshold choice.

Alternative Method / Trick

Precision-Recall Curve (PR Curve) as an Alternative:
When positive examples are very rare (high class imbalance), the ROC curve can be overly optimistic. A **Precision-Recall curve** (plots Precision vs. Recall) is more informative in such cases. Choose:

- ROC/AUC — when class distribution is roughly balanced.
- PR curve / AUC-PR — when dealing with rare positive events (fraud, disease, spam).

5 Underfitting and Overfitting

5.1 Definitions

Models can fail in two fundamentally different ways:

Property	Underfitting	Overfitting
Hypothesis class	Too simple for data	Too complex for data
Training error	High	Low (near 0)
Test/Validation error	High	High
Generalisation	Poor	Poor (different reason)

Key Idea

The goal: Find a model complex enough to fit the training data well, but simple enough to *generalise* to new, unseen data. This is the central tension in machine learning.

5.2 Underfitting

An **underfitted model** uses a hypothesis class that is too simple to capture the patterns in the data.

Example — Housing Price Regression: Fitting a straight line (degree-1 polynomial) to data that follows a curved pattern. The line cannot capture the curvature, yielding large errors on both training and test data.

Characteristics:

- High training loss *and* high test loss.
- Model has *high bias* — it presupposes a structure (e.g. linearity) that the data does not support.
- Adding more training data does not help much; the model is fundamentally limited by its hypothesis class.

Fixes:

- Use a more complex model (higher-degree polynomial, more layers in a neural network).
- Add more relevant features.
- Reduce regularisation.

5.3 Overfitting

An **overfitted model** uses a hypothesis class that is so complex it memorises the training data including its noise, and fails to generalise.

Example — Housing Price Regression: Fitting a degree-9 polynomial to 10 data points. The curve passes exactly through every training point (zero training loss) but oscillates wildly between points, giving very large errors on new data.

Characteristics:

- Very low (or zero) training loss but high test loss.
- Model has *high variance* — it is overly sensitive to the specific training examples it saw.
- Adding more training data *does* help; overfitting is often a symptom of too little data relative to model complexity.

Fixes:

- Use a simpler model.
- Collect more training data.
- Apply regularisation (ℓ_1 or ℓ_2).
- Use dropout (for neural networks).
- Cross-validation to detect early.

5.4 Bias and Variance

Two key statistical properties explain under/overfitting:

Property	Bias	Variance
Cause	Limited model flexibility	Sensitivity to training data
Result	Systematic error	Erratic error
Training error	High	Low
Test error	High (similar to train)	High (much larger than train)
Hypothesis class	Too simple	Too complex
Leads to	Underfitting	Overfitting

Key Idea

Bias-Variance Tradeoff:
Total expected test error $\approx$ Bias^2 + Variance + Noise.
Increasing model complexity:

- Decreases bias (model can fit more patterns).
- Increases variance (model becomes more sensitive to noise).

The optimal model complexity is where the sum Bias^2 + Variance is minimised. This is not a flaw in ML — it is an *inherent statistical tradeoff* that every model selection decision must negotiate.

Real-Life Analogy: Imagine studying for an exam.

- **Underfitting (high bias):** You barely studied. You do poorly on every question — training (practice exam) and real exam alike.
- **Overfitting (high variance):** You memorised the exact questions from the practice exam word-for-word. You ace the practice exam but fail when the real exam asks slightly differently-worded questions.
- **Good generalisation:** You understood the *concepts* from the practice exam and apply them to novel questions.

5.5 Regression Example (Polynomial Fitting)

Example

Given housing price data with $N = 10$ points:

1. **Degree-1 polynomial (underfitting):** A straight line. Misses the curvature in data. High training *and* test error. High bias.

$$h(x) = w_0 + w_1 x$$

2. **Degree-2 polynomial (good fit):** A quadratic curve. Captures the main trend without memorising noise. Low training error and low test error. Model generalises well.

$$h(x) = w_0 + w_1 x + w_2 x^2$$

3. **Degree-9 polynomial (overfitting):** A degree-9 curve passes through all 10 training points exactly (zero training loss). Wildly oscillates between training points. Extremely high test error. High variance.

$$h(x) = w_0 + w_1 x + \cdots + w_9 x^9$$

5.6 Classification Example

The same principles apply in classification:

- A **linear decision boundary** (e.g. logistic regression without feature engineering) may underfit data that requires a curved boundary.
- A decision tree with unlimited depth will memorise every training point, overfitting the training data.
- A decision tree with an appropriate depth (pruned, cross-validated) will generalise well.

Visual rule of thumb:

- Decision boundary that is too *rigid and straight* when data is not linearly separable $\Rightarrow$ underfitting.
- Decision boundary that follows every training point with complex wiggles $\Rightarrow$ overfitting.
- Smooth boundary that captures the main separation without chasing every point $\Rightarrow$ good generalisation.

6 Practice Questions

Practice Questions

Topic: Multiclass Classification Strategies

Q1 (Easy): You have a 5-class classification problem. How many binary classifiers does One-vs-Rest require? How many does One-vs-One require?
Answer: OvR: $K = 5$ classifiers. OvO: $K(K-1)/2 = 5 \times 4/2 = 10$ classifiers.

Q2 (Medium): You are building a multiclass classifier for 100 object categories (like ImageNet). Which strategy (OvR or OvO) is more practical, and why?
Answer: **OvR** is more practical. OvO would require $100 \times 99/2 = 4950$ classifiers, each stored in memory and evaluated at inference — prohibitively expensive. OvR only requires 100 classifiers. Additionally, modern deep learning uses softmax (multinomial logistic regression) directly, bypassing the need for either strategy.

Topic: Softmax and Multinomial LR

Q3 (Easy): Compute the softmax of the score vector $\mathbf{s} = [3,\ 1,\ 0]$.
Answer:
$Z = e^3 + e^1 + e^0 = 20.09 + 2.72 + 1.00 = 23.81$

$$\mathbf{p} = \left[\frac{20.09}{23.81}, \frac{2.72}{23.81}, \frac{1.00}{23.81}\right] \approx [0.844,\ 0.114,\ 0.042]$$

Prediction: class 1.

Q4 (Exam-level): Show that when $K = 2$, the softmax function with score vector $[s,\ 0]$ reduces to the sigmoid function.
Proof:

$$\text{softmax}([s, 0])_1 = \frac{e^s}{e^s + e^0} = \frac{e^s}{e^s + 1} = \frac{1}{1 + e^{-s}} = \sigma(s)$$

□

Topic: Evaluation Metrics

Q5 (Easy): A classifier on 100 examples produces: TP=40, FP=10, FN=20, TN=30. Compute accuracy, precision, recall, and F1 score.
Answer:

$$\begin{aligned}
\text{Accuracy} &= (40 + 30)/100 = 70\% \\
\text{Precision} &= 40/(40 + 10) = 80\% \\
\text{Recall} &= 40/(40 + 20) = 66.7\% \\
F_1 &= 2(0.80)(0.667)/(0.80 + 0.667) = 72.7\%
\end{aligned}$$

Q6 (Medium): A medical AI detects a rare disease that affects 1% of the population. After testing 10,000 patients, the model correctly identifies 80 of the 100 sick patients but also flags 500 healthy patients as sick. Compute precision and recall. Which metric is more important here?
Answer: TP = 80, FN = 20, FP = 500.
Precision = $80/(80+500) = 13.8\%$ (very poor)
Recall = $80/(80+20) = 80\%$
Recall is more important: missing a sick patient (FN) is far more dangerous than further testing of healthy patients (FP). The model is acceptable in recall but its precision is poor — many unnecessary follow-up tests. A better system needs to balance both, e.g. through threshold tuning.

Topic: Bias, Variance, Underfitting, Overfitting

Q7 (Medium): A model achieves 98% training accuracy and 55% test accuracy. Is this underfitting or overfitting? What is the bias/variance situation?
Answer: **Overfitting.** Low training error + high test error $\Rightarrow$ high variance. Low bias (model fits the training data well) but high variance (model is too sensitive to training data specifics).
Remedies: collect more training data, simplify the model, add regularisation, use dropout (for neural networks), perform cross-validation.

Q8 (Easy): A model achieves 60% training accuracy and 58% test accuracy on a problem where random guessing gives 50% accuracy. What is happening?
Answer: **Underfitting.** Both training and test errors are high and close to each other. This indicates high bias — the model is too simple.
Remedies: use a more complex hypothesis class, add features, or reduce regularisation.

7 Key Takeaways — Week 4

Quick Summary

- **Logistic regression GD update** (from worked example): $(h^{(i)}_{\text{old}} - y^{(i)})\boldsymbol{\phi}^{(i)}$ accumulated and multiplied by α/N.
- **One-vs-Rest:** Train K binary classifiers on full dataset. Predict with $\arg\max$ of confidence scores.
- **One-vs-One:** Train $K(K-1)/2$ classifiers on class-pair subsets. Predict by majority vote.
- **Multinomial LR:** Weight matrix $\mathbf{W}$ of order $(d+1) \times K$; score $\mathbf{S} = \mathbf{XW}$; probabilities via row-wise softmax.
- **Softmax:** $P_{ik} = e^{s_{ik}} / \sum_j e^{s_{ij}}$; generalises sigmoid to K classes.
- **Categorical cross entropy loss** (multinomial): $-\frac{1}{N}\sum_i \sum_k Y_{ik} \log P_{ik}$. Gradient $= \frac{1}{N}\mathbf{X}^\top(\mathbf{P} - \mathbf{Y})$.
- **Confusion matrix** cells: TP, FP, FN, TN. Accuracy $= (TP + TN)/\text{total}$ —

misleading under imbalance.

- **Precision** (cost of FP) $= TP/(TP + FP)$. **Recall** (cost of FN) $= TP/(TP + FN)$.
- **F1 score** = harmonic mean of precision and recall; balances both.
- **ROC curve:** TPR vs. FPR at varying thresholds. **AUC:** area under ROC; 1 = perfect, 0.5 = random.
- **Underfitting:** high bias. Simple model, high training AND test error. Fix: more complexity.
- **Overfitting:** high variance. Complex model, low training error, high test error. Fix: regularise, more data.
- **Bias-variance tradeoff:** Test error $\approx$ Bias^2 + Variance + Noise. Cannot minimise both simultaneously.